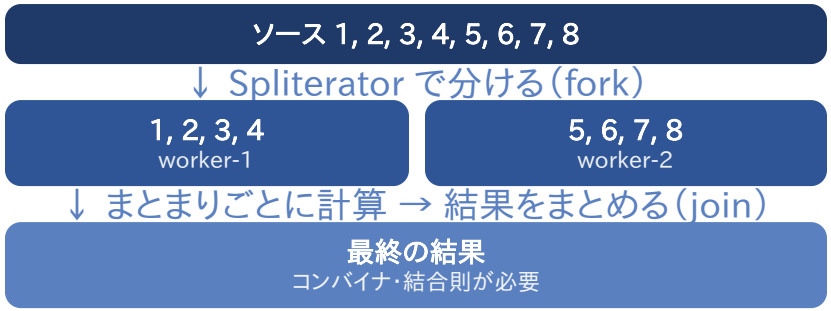
Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 3 Java ストリーム API　#09

並列ストリーム

parallelStream・parallel、順序と forEachOrdered、結合則・単位元・副作用

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java SE 17 API の java.util.stream パッケージの説明・BaseStream・Stream・Collectors・ForkJoinPool、The Java Tutorials の Parallelism）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21）で何度か実行して確かめた

全体像 分けて、同時に処理して、まとめる



書き方	結果
list.parallelStream()	並列（Collection のメソッド）
stream.parallel()	並列
s.parallel().sequential()	逐次（最後が効く）
s.sequential().parallel()	並列
Stream.of(1).unordered()	逐次のまま

Stream.of(...).parallelStream() は cannot find symbol。

スレッド：main と ForkJoinPool.commonPool-worker-N。

出典：API BaseStream.parallel・sequential・isParallel・unordered、Collection.parallelStream、ForkJoinPool.commonPool／JDK 17 でのコンパイルと実行の結果

第1章

順序

守る操作と、守らない操作。

順序を守る操作・守らない操作

操作	並列での順序
forEach	決まらない
forEachOrdered	出現順序どおり
collect(toList())・toList()	出現順序どおり
joining・順序のある reduce	出現順序どおり
findFirst	必ず最初
findAny	決まらない

> 出現順序（encounter order）：List・配列・range は持つ、HashSet は持たない。

```
IntStream.rangeClosed(1, 8).parallel()
  .forEach(System.out::print);
// 例：65782341（毎回変わる）
IntStream.rangeClosed(1, 8).parallel()
  .forEachOrdered(sb::append);
// 12345678（毎回同じ）
List.of(3,1,2).parallelStream().map(n -> n * 2).toList()
// [6, 2, 4]
List.of(3,1,2).parallelStream().sorted()
  .forEach(System.out::print);
// 順序は決まらない
```

出典：API Stream.forEach・forEachOrdered・findFirst・findAny・toList、java.util.stream パッケージの説明「Ordering」／JDK 17 で複数回実行した結果

第2章

並列で正しく動かす条件

結合則・単位元・副作用・干渉。

単位元と結合則

単位元でない値を渡すと

```
Stream.of(1,2,3).reduce(10, Integer::sum)            // 16
Stream.of(1,2,3).parallel().reduce(10, Integer::sum) // 36
// (10+1) + (10+2) + (10+3)
```

結合則を満たさない演算

```
IntStream.rangeClosed(1, 8).reduce(0, (x, y) -> x - y)
// -36
IntStream.rangeClosed(1, 8).parallel().reduce(0, (x, y) -> x - y)
// 0（JDK 17 での例。-36 にならない）
```

演算	結合則	単位元
足し算	○	0
掛け算	○	1
max・min	○	―（Optional 版を使う）
文字列の連結	○	""
引き算・割り算	×	―

> コンパイラも実行時も調べない。黙って違う結果になる。

出典：API java.util.stream パッケージの説明「Associativity」「Reduction operations」、Stream.reduce／JDK 17 で 5 回実行した結果

副作用と、安全な集め方

```
List<Integer> l = new ArrayList<>();
IntStream.range(0, 10000).parallel().forEach(l::add);
// × 要素が欠ける・ArrayIndexOutOfBoundsException

List<Integer> s = Collections.synchronizedList(new ArrayList<>());
IntStream.range(0, 10000).parallel().forEach(s::add);
// △ 10000 個（順序は決まらない、遅い）

List<Integer> r = IntStream.range(0, 10000).parallel()
    .boxed().collect(Collectors.toList());
// ○ 10000 個・元の順
```

求められること	中身
干渉しない	実行中にソースを変えない
状態を持たない	外の Set などに頼らない（重複除去は distinct）
副作用を避ける	外のコレクションに書かず collect で作る

> forEachOrdered は 1 つずつ順に処理するので、
> StringBuilder にも安全に append できる。

出典：API java.util.stream パッケージの説明「Non-interference」「Stateless behaviors」「Side-effects」、Collections.synchronizedList／JDK 17 で 20 回実行した結果

3 引数の reduce と並行コレクタ

```
Stream.of("a","bb","cc").parallel()
  .reduce(0,
          (n, s) -> n + s.length(),   // accumulator
          Integer::sum)                // combiner
// 5（並列でも同じ）
```

> combiner は並列のときだけ呼ばれる。まちがっていても逐次では気づかない。

```
ConcurrentMap<Integer, List<String>> m =
  Stream.of("a","bb","cc").parallel()
    .collect(Collectors.groupingByConcurrent(String::length));
// m.get(2) は bb と cc（順は決まらない）
// m.getClass() → ConcurrentHashMap
```

	groupingBy	groupingByConcurrent
並列での集め方	まとまりごとに Map → まとめる	1 つの ConcurrentMap に直接
順序	守る	守らない

出典：API Stream.reduce(U, BiFunction, BinaryOperator)、Collectors.groupingByConcurrent・toConcurrentMap／JDK 17 での実行結果

本番での問われ方

ねらい	見るところ
並列かどうか	parallel・sequential は最後が効く、parallelStream は Collection だけ、unordered は並列にしない
出力が決まるか	forEach・findAny は決まらない、forEachOrdered・toList・findFirst・joining は決まる
reduce の結果	単位元でない値・結合則を満たさない演算は並列で変わる
副作用	ArrayList への add は危険、collect を使う
速さ	要素が少ない・処理が軽いと並列のほうが遅いことがある

出典：試験内容チェックリスト 3／本資料の問題演習（全 30 問、JDK 17 で確認）